

MODELAGEM DE BANCO DE DADOS APLICADA À ECONOMIA DO CUIDADO:

Proposta de um sistema de gestão para a AM Consultoria¹

DATABASE MODELING APPLIED TO THE CARE ECONOMY: Proposal for a management system for AM Consultoria

Cássia Irene Serra Nascimento²

Leonardo Ferreira Souza³

Melissa Mendes Wolff Ribeiro⁴

Felipe Gomes Barbosa⁵

RESUMO

Propor um modelo de banco de dados relacional para a AM Consultoria, empresa fictícia especializada em gestão operacional de instituições da economia do cuidado, tomada como cenário simulado. Levantar os requisitos por meio de sessões estruturadas com um agente conversacional emulando o proprietário, cobrindo contratos com histórico de alterações, controle de visitas e projetos, acompanhamento de pendências e geração de faturamento. Construir um modelo com quinze entidades aplicando normalização, integridade referencial e versionamento por *Slowly Changing Dimension Type 2*, a partir de um processo iterativo que incluiu revisão do modelo lógico inicial. Implementar o esquema em PostgreSQL com *migrations* versionadas e validar os fluxos operacionais com quatorze consultas SQL e integração com *backend* em Python e interface *frontend*. Concluir que a estrutura proposta cobre os principais fluxos da consultoria, com destaque para o versionamento contratual e o indicador de saúde operacional por cliente, e que o modelo não contempla ainda cenários de múltiplos consultores nem foi validado com um usuário real.

¹ Short Paper apresentado à disciplina Banco de Dados do Curso de Engenharia de Software do Centro Universitário Unidade de Ensino Superior Dom Bosco - UNDB.

² Graduanda do 3º Período do Curso de Engenharia de Software do Centro Universitário Unidade de Ensino Superior Dom Bosco - UNDB. E-mail: 002-029368@aluno.undb.edu.br.

³ Graduando do 3º Período do Curso de Engenharia de Software do Centro Universitário Unidade de Ensino Superior Dom Bosco - UNDB. E-mail: 002-029373@aluno.undb.edu.br.

⁴ Graduanda do 3º Período do Curso de Engenharia de Software do Centro Universitário Unidade de Ensino Superior Dom Bosco - UNDB. E-mail: 002-029377@aluno.undb.edu.br.

⁵ Professor Mestre. Docente do Curso de Engenharia de Software do Centro Universitário Unidade de Ensino Superior Dom Bosco - UNDB. E-mail: felipe.barbosa@undb.edu.br.

Palavras-chave: Banco de dados relacional. Modelagem de dados. Sistema de gestão. Economia do cuidado.

ABSTRACT

To propose a relational database model for AM Consultoria, a fictional consulting firm specializing in operational management for care economy institutions, treated as a simulated scenario. To elicit requirements through structured sessions with a conversational agent emulating the business owner, covering contract management with revision history, visit and project tracking, pending task management, and billing generation. To build a fifteen-entity model applying normalization, referential integrity, and *Slowly Changing Dimension Type 2* versioning, through an iterative process that included revision of an initial logical model. To implement the schema in PostgreSQL with versioned *migrations* and validate operational flows using fourteen SQL queries and integration with a Python *backend* and web *frontend*. To conclude that the proposed structure covers the consultancy's core operational flows, highlighting contract versioning and the per-client operational health indicator, and that the model does not yet address multi-consultant scenarios or validation with an actual user.

Keywords: Relational database. Data modeling. Management system. Care economy.

1 INTRODUÇÃO

Segundo Sousa (2017), a economia do cuidado compreende o conjunto de atividades voltadas à satisfação de necessidades físicas ou psicológicas de terceiros, incluindo cuidado de crianças, jovens e pessoas dependentes, exercidas de forma remunerada ou não. Na prática, isso abrange clínicas de reabilitação, centros de apoio a pessoas com deficiência e organizações de assistência social, contextos que compartilham um desafio de gestão específico: a heterogeneidade dos serviços prestados dificulta a padronização dos registros e a produção de informações confiáveis a partir deles.

Instituições da economia do cuidado compartilham uma característica que complica a gestão operacional: a prestação de serviço é contínua, personalizada e sujeita a variações frequentes de escopo e condições contratuais. Diferente de setores com serviços padronizados, cada cliente tende a demandar combinações específicas de frequência, tipo de atendimento e forma de cobrança, o que dificulta a criação de registros uniformes e a consolidação de informações ao longo do tempo. Essa

heterogeneidade torna os sistemas de informação especialmente relevantes nesse contexto: sem uma estrutura de dados capaz de acomodar variação sem perder rastreabilidade, qualquer tentativa de digitalização reproduz no sistema os mesmos problemas que existiam nos registros em papel.

A AM Consultoria é uma empresa fictícia que presta serviços de diagnóstico e melhoria organizacional a instituições desse setor. Suas operações envolvem visitas periódicas, projetos estruturados, contratos com variações ao longo da vigência e acompanhamento de pendências operacionais. No cenário simulado, todas essas atividades eram controladas de forma dispersa, em cadernos, mensagens de aplicativos e planilhas avulsas. O efeito prático era direto: cobranças sem registro formal, pendências acumuladas sem responsável definido e histórico de clientes impossível de reconstruir sem busca manual em múltiplas fontes.

Resolver esse problema exige mais do que digitalizar registros: é necessário estruturá-los de forma que possam ser cruzados de maneira confiável. A modelagem de banco de dados é a representação formal das entidades, atributos e relacionamentos de um domínio em estruturas persistentes e organizadas. Segundo Heuser (2022), esse processo se desdobra em três etapas: a modelagem conceitual, que captura requisitos independentemente de tecnologia; o modelo lógico, que os traduz para estruturas do paradigma relacional; e o modelo físico, que define como os dados serão armazenados em um SGBD específico. Para a AM Consultoria, esse processo partiu de um requisito central: fazer com que contratos, visitas e cobranças pudessem ser cruzados de forma confiável.

A consistência desse cruzamento depende de dois mecanismos: normalização, que elimina dependências funcionais indevidas, e integridade referencial, que impede vínculos inconsistentes entre tabelas (HEUSER, 2022). Sem esses dois mecanismos funcionando juntos, seria impossível rastrear o que foi cobrado em cada período de um contrato que teve o valor alterado no meio da vigência.

O objetivo deste trabalho é propor um modelo de banco de dados que sirva como estrutura central para a gestão da AM Consultoria, permitindo que os fluxos operacionais identificados possam ser executados com consistência e rastreabilidade. O problema concreto que o modelo deve resolver é direto: sem um esquema de dados

formal, é impossível gerar faturamento confiável ou reconstruir o histórico de um cliente sem esforço manual.

Como objetivos específicos, busca-se: a) levantar os requisitos do negócio por meio de sessões estruturadas com um agente conversacional simulando o proprietário; b) construir os modelos conceitual, lógico e físico; c) implementar o modelo em PostgreSQL com *migrations* versionadas; e d) validar a solução por integração com camadas de *backend* e interface.

O trabalho está organizado em quatro seções: a próxima descreve o processo metodológico adotado; a seção 3 apresenta e discute as decisões de modelagem; e a seção 4 reúne as considerações finais e limitações.

2 METODOLOGIA

O processo foi conduzido em quatro etapas sequenciais, com revisões iterativas entre elas. Na primeira etapa, realizaram-se sessões de levantamento de requisitos por simulação operacional, em que o papel do proprietário foi desempenhado por um agente conversacional (ChatGPT), configurado para emular padrões realistas de tomada de decisão. A equipe elaborou perguntas sobre gestão de contratos, formas de cobrança, controle de visitas e projetos, acompanhamento de pendências e faturamento. O formato exigiu atenção na formulação das perguntas e capacidade de interpretar respostas por vezes ambíguas. Vale registrar que o agente simulado substituiu, mas não eliminou, a necessidade de validação com um usuário real, limitação discutida nas considerações finais.

As sessões foram organizadas em três blocos temáticos: o primeiro cobriu a estrutura de clientes e contratos, incluindo variações de escopo e formas de pagamento; o segundo tratou do controle operacional, com foco em visitas, projetos e pendências; e o terceiro abordou faturamento e histórico. Em cada bloco, a equipe formulou perguntas abertas e registrou as respostas em linguagem natural antes de traduzi-las para requisitos formais.

Quando o agente retornava respostas ambíguas, por exemplo ao definir o que constituía uma “Visita”, que na rotina do cliente englobava desde reuniões rápidas de urgência até intervenções de crise complexas, mas recebia o mesmo registro genérico em papel, a questão era reformulada até que a regra de negócio ficasse clara

o suficiente para gerar uma decisão de modelagem: criar a entidade visitas com atributo tipo_visita restrito por CHECK, capaz de absorver essas variações sem multiplicar entidades. Esse processo gerou aproximadamente trinta requisitos funcionais distribuídos entre as quatro áreas temáticas, que foram revisados em conjunto antes da etapa de modelagem conceitual.

Na segunda etapa, as informações levantadas foram organizadas em um modelo conceitual descrevendo entidades, atributos e relacionamentos em linguagem natural, sem definição de tipos de dados ou chaves. Esse modelo serviu de base para um Diagrama de Entidade-Relacionamento, representando a versão inicial do modelo lógico. Ao iniciar a implementação física, surgiram dois problemas que exigiram retornar às camadas anteriores.

O primeiro envolveu a tabela faturamento_mensal: o campo valor_total estava configurado como gerado, calculado dinamicamente a partir de outros dados. O problema é que esse cálculo poderia variar caso o contrato fosse alterado após a emissão da fatura, o que inviabilizaria a auditoria de valores históricos. O segundo problema era que mes e ano eram colunas separadas, o que dificultava os agrupamentos nas consultas de receita mensal. A solução foi armazenar valor_total no momento da geração do faturamento e unificar mes e ano em um único campo mes_ano com uma *constraint* garantindo sempre o primeiro dia do mês como referência temporal. Esse ciclo de modelar, implementar, identificar problemas e corrigir as duas camadas ocorreu mais de uma vez antes de o esquema estabilizar.

A terceira etapa consistiu em desenvolver consultas SQL representando os fluxos operacionais principais: agenda diária, histórico de contratos, faturamento mensal consolidado, indicador de saúde operacional por cliente e rastreamento de pendências. Essas consultas funcionaram tanto como validação do modelo quanto como prototipagem dos fluxos esperados.

Na quarta etapa, o modelo foi integrado a uma aplicação com *backend* em Python e interface *frontend*. Para que a validação cobrisse condições próximas de uso real, o banco foi hidratado com dados simulando seis meses de operação da consultoria mais trinta dias de registros futuros, permitindo testar não apenas a integridade estrutural do esquema, mas também o comportamento das consultas diante de um histórico operacional representativo.

3 RESULTADOS E DISCUSSÃO

3.1 MAPEAMENTO DE REQUISITOS E REGRAS DE NEGÓCIO

Tendo em vista que todas as operações da AM Consultoria são voltadas aos seus clientes, o levantamento de requisitos teve como foco inicial as instituições atendidas. Atualmente, a empresa possui sete clientes, cada um com especificidades como tipo de instituição, contatos internos, nível de complexidade, contratos firmados, serviços acordados, visitas realizadas, projetos executados, formas de pagamento, pendências, faturamento e histórico operacional.

Cada cliente pode possuir múltiplos contatos, sendo obrigatório registrar nome, papel na organização (decisor operacional, financeiro ou de emergência) e meios de contato. Os contratos definem o escopo do atendimento e compartilham uma estrutura base, incluindo serviços, frequência de acompanhamento e formas de pagamento, que podem ser múltiplas. Também é necessário o controle de histórico contratual, em que alterações em contratos vigentes geram o encerramento do registro anterior e a criação de uma nova versão atualizada.

Os serviços da consultoria se organizam principalmente em visitas e projetos. As visitas constituem o principal meio de acompanhamento, sendo necessário registrar data, hora, tipo_visita (rotineira, urgente, pontual, estruturada e acompanhamento direcionado), descrição das atividades e pendências geradas. Em visitas extras, deve-se registrar também o solicitante. As pendências podem ou não estar vinculadas a visitas, devendo conter descrição, responsável (cliente ou consultor) e prazo para resolução.

Os projetos representam iniciativas estruturadas com início, meio e fim, envolvendo ações, reuniões e entregas. Devem conter título, descrição, datas de início e prazo, além de valor total e possibilidade de pagamento parcelado. Projetos extras também devem registrar solicitante e aprovador. Além das atividades previstas, a consultoria lida com eventos críticos, atualmente não formalizados, mas relevantes para análise operacional. Quando registrados, devem conter descrição do ocorrido, data e ação tomada.

A análise dos requisitos evidencia que os principais problemas não decorrem apenas da falta de digitalização, mas da ausência de uma estrutura central

capaz de relacionar contratos, visitas, projetos e faturamento de forma consistente ao longo do tempo. Isso torna a rastreabilidade histórica um requisito central da modelagem, especialmente em cenários de alteração contratual, cobranças extraordinárias e acompanhamento de pendências em diferentes contextos.

3.2 MODELO CONCEITUAL

Nesta etapa, analisaram-se os requisitos levantados e identificaram-se quinze entidades para compor o banco de dados da AM Consultoria: clientes, contatos, contratos, tipo_pagamento, contrato_pagamento, historico_contrato, projetos, projetos_parcela, projetos_extra, entregas, visitas, visitas_extra, pendencias, eventos_criticos e faturamento_cliente. As relações entre as entidades de negócio e suas respectivas regras de cardinalidade são detalhadas na Tabela 1.

Tabela 1 – Dicionário de relacionamentos e cardinalidades do modelo conceitual.

ENTIDADE 1	RELACIONAMENTO	ENTIDADE 2
CLIENTES (0, n)	possui	(1,1) CONTATOS
CLIENTES (0, n)	firma	(1,1) CONTRATOS
CONTRATO_PAGAMENTO (1, 1)	pertence a	(0, n) CONTRATOS
HISTORICO_CONTRATOS (1, 1)	registra transição de	(0, 1) CONTRATOS
PROJETOS (1, 1)	vincula-se a	(0, n) CONTRATOS
VISITAS (1, 1)	ocorre dentro de	(0, n) CONTRATOS
FATURAMENTO_CLIENTE (1, 1)	pertence a	(0, n) CONTRATOS
EVENTOS_CRITICOS (1, 1)	associa-se a	(0, n) CONTRATOS
PENDENCIAS (1, 1)	pertence a	(0, n) CONTRATOS
TIPOS_PAGAMENTO (0, n)	classifica	(1, n) CONTRATO_PAGAMENTO
PROJETOS (0, n)	é dividido em	(1, 1) PROJETO_PARCELAS
PROJETOS (0, 1)	pode se classificar como	(1, 1) PROJETOS_EXTRA
ENTREGAS (1, 1)	associa-se a	(0, n) PROJETOS
VISITAS (0, 1)	pode ocorrer dentro de	(0, n) PROJETOS
VISITAS (0, 1)	pode se classificar como	(1, 1) VISITAS_EXTRA
VISITAS (0, n)	pode gerar	(0, 1) PENDENCIAS
VISITAS (0, n)	pode gerar	(0, 1) EVENTOS_CRITICOS
EVENTOS_CRITICOS (0, 1)	pode ocorrer durante	(0, n) VISITAS
CONTATOS (0, n)	solicita	(1,1) VISITAS_EXTRA
CONTATOS (0, n)	solicita	(1, 1) PROJETOS_EXTRA

CONTATOS (0, n)	aprova	(0, 1) PROJETOS_EXTRA
-----------------	--------	-----------------------

Fonte: Autores (2026).

A centralidade da entidade contratos no modelo reflete a própria lógica operacional da consultoria, já que visitas, projetos, pendências e cobranças só fazem sentido quando vinculados a um contrato vigente. Como destacam Elmasri e Navathe (2018), a integridade referencial garante que não existam registros sem uma âncora válida, princípio que orientou essa decisão: sem o vínculo obrigatório com contratos, não seria possível atribuir eventos a clientes de forma confiável.

O versionamento contratual foi implementado pela entidade `historico_contratos`, que registra cada transição entre um contrato encerrado e seu substituto. Esse padrão corresponde ao *SCD Type 2, Slowly Changing Dimension Type 2*, no qual cada alteração gera uma nova versão ativa enquanto o registro anterior é preservado, não sobrescrito (IBM, [s.d.]). Com isso, é possível reconstruir o estado exato de qualquer contrato em qualquer período passado.

O faturamento foi estruturado em dois módulos: `faturamento_cliente` para cobranças mensais recorrentes e `projeto_parcelas` para parcelas de projetos com valor e datas próprias. Misturá-los em uma única tabela complicaria as consultas de receita mensal, que precisam somar fontes distintas, o que a Q05 resolve via UNION ALL.

A decisão de separar entidades principais (visitas, projetos) de complementares (`visitas_extra`, `projetos_extra`) evitou que atributos exclusivos de situações excepcionais, como o campo `solicitante`, que só existe em visitas fora do contrato, ficassem nulos na maioria dos registros. Manter esse campo diretamente na tabela `visitas` significaria que praticamente todos os registros rotineiros carregariam um atributo sem valor nem significado.

Pendências e eventos críticos foram vinculados obrigatoriamente a contratos e opcionalmente a visitas, garantindo rastreabilidade mínima por cliente mesmo quando a ocorrência surge fora de um atendimento formal.

O principal desafio durante a modelagem foi decidir o que pertencia à entidade principal e o que devia ir para uma entidade satélite. No início, a tendência foi colocar campos opcionais diretamente nas entidades de visitas e projetos, até que ficou claro que isso criaria registros com muitos campos nulos e tornaria as consultas

3.3 MODELO LÓGICO

Figura 1 - Diagrama do modelo lógico do banco de dados relacional.⁶



⁶ Disponível para melhor visualização em:
<https://drive.google.com/drive/folders/1KTSrJD9KMI55ChHpVad3MeME7Synobae?usp=sharing>

As quinze tabelas do modelo lógico foram estruturadas de forma a reduzir redundâncias e preservar a integridade referencial entre os registros operacionais da consultoria. A tabela `historico_contratos` utiliza pares de chaves estrangeiras apontando para o contrato encerrado e para o novo contrato correspondente, permitindo reconstruir a sequência de alterações contratuais sem sobrescrever informações anteriores. Essa abordagem mantém o histórico íntegro e rastreável ao longo do tempo.

Essa estratégia foi considerada mais adequada do que a simples atualização de registros existentes, pois evita perda de contexto histórico em situações nas quais valores, frequência de visitas ou formas de cobrança sejam alterados durante a vigência da relação contratual. Em termos operacionais, isso permite reconstruir o estado exato de um contrato em períodos anteriores, característica importante para auditoria, análise financeira e resolução de divergências administrativas.

Os atributos `valor_total` e `visitas_realizadas`, presentes em `faturamento_cliente`, foram armazenados intencionalmente embora sejam deriváveis de outras operações do sistema. No modelo lógico inicial, `valor_total` estava configurado como campo gerado, calculado automaticamente a partir de outros valores.

Essa abordagem foi revisada quando se percebeu que um campo calculado dinamicamente retornaria valores diferentes dependendo do estado atual das tabelas, tornando impossível auditar o que foi efetivamente cobrado em um mês anterior se o contrato tivesse sido alterado desde então. A decisão de armazenar o valor no momento da geração da fatura é uma desnormalização controlada, que viola a Terceira Forma Normal, mas tem justificativa operacional clara: em contextos financeiros, o histórico precisa ser imutável após emissão.

Relacionamentos opcionais, como o vínculo entre visitas e projetos e entre pendências e visitas, foram implementados com chaves estrangeiras *nullable* diretamente nas tabelas dependentes. A alternativa seria criar tabelas associativas intermediárias, mas dado que a opcionalidade é genuína nos dois casos (há visitas que não pertencem a nenhum projeto, e há pendências sem visita de origem), a tabela intermediária adicionaria complexidade às consultas sem ganho real de integridade.

As decisões de tipo de dado no modelo lógico também refletiram restrições operacionais. O campo `tipo_visita` em `visitas` foi definido como tipo enumerado, restringindo os valores possíveis a “rotineira”, “urgente”, “pontual”, “estruturada” e “acompanhamento direcionado”, o que permitiu que a Q11 filtrasse visitas de emergência sem depender de comparações com strings livres.

O campo `mes_ano` em `faturamento_cliente` foi definido como DATE em vez de colunas separadas ou VARCHAR, decisão que simplificou os agrupamentos em consultas de receita mensal e eliminou a necessidade de conversão a cada consulta. Essas escolhas mostram que a passagem do modelo lógico para o físico não foi apenas uma tradução técnica: parte das decisões de tipo e restrição emergiu da antecipação de como as consultas seriam escritas.

3.4 MODELO FÍSICO E IMPLEMENTAÇÃO

Para a implementação física, optou-se pelo PostgreSQL. A escolha não foi apenas por conveniência: o sistema oferece suporte consistente a *constraints* declarativas, transações e *migrations* versionadas, recursos que se mostraram necessários ao longo do projeto para evoluir o esquema sem reestruturações completas. Silberschatz, Korth e Sudarshan (2020) destacam que a implementação física de um banco relacional deve considerar não apenas armazenamento e desempenho, mas também mecanismos capazes de garantir integridade, concorrência e confiabilidade operacional.

A estrutura física foi organizada em quinze arquivos de *migration*, cada um responsável pela criação de uma tabela⁷. Essa convenção garante que o banco possa ser recriado de forma reproduzível a partir do zero, que a ordem de criação das tabelas respeite as dependências de chaves estrangeiras, e que a evolução do esquema ao longo do tempo seja rastreável no controle de versão. O versionamento foi implementado por convenção de nomenclatura nos próprios nomes dos arquivos, e não por uma ferramenta automatizada como Flyway ou Alembic, o que é suficiente para o porte do projeto.

⁷ Os arquivos de *migration*, assim como o projeto completo, está disponível na íntegra em: <https://github.com/Cassia-Irene/AM-Consultoria>

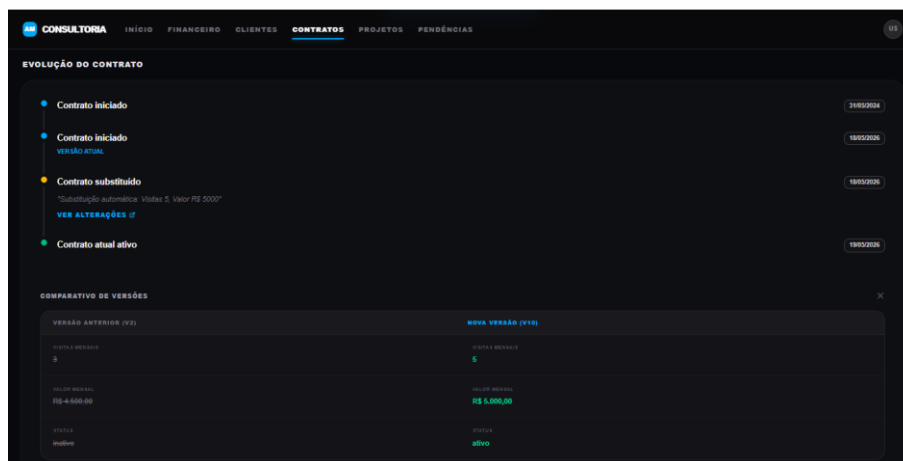
A consistência das regras do modelo foi reforçada diretamente na camada de banco de dados por meio de restrições declarativas implementadas no DDL. Na tabela `faturamento_cliente`, por exemplo, a restrição CHECK (`EXTRACT(DAY FROM mes_ano) = 1`) garante a padronização do registro mensal de faturamento sempre utilizando o primeiro dia do mês como referência temporal. Já a restrição UNIQUE (`id_contrato, mes_ano`) impede a duplicidade de registros de faturamento para um mesmo contrato no mesmo período.

De forma semelhante, a tabela `historico_contratos` utiliza a restrição CHECK (`id_contrato_encerrado <> id_contrato_novo`) para impedir registros inválidos em que um contrato seja associado a si próprio como substituição. Com isso, parte da validação das regras críticas do sistema permanece garantida diretamente pelo banco de dados, independentemente da camada de aplicação. Colocar essas validações no banco, e não só no *backend*, foi uma decisão deliberada, pois, se alguém inserir dados diretamente no PostgreSQL, o que pode acontecer em manutenção ou *debugging*, as regras continuam valendo.

Para validar que o modelo responde às necessidades operacionais identificadas no levantamento de requisitos, foram desenvolvidas quatorze consultas SQL executadas sobre o banco hidratado com dados representativos: seis meses de operação simulada da consultoria, cobrindo contratos com diferentes históricos de alteração, visitas rotineiras e extras, projetos em andamento e encerrados, faturamentos mensais e pendências em aberto, mais trinta dias de registros futuros para testar os fluxos de agenda e vencimento. Três consultas merecem destaque pela complexidade e pela correspondência direta com os problemas relatados.

A primeira, Q06, reconstrói o histórico de alterações contratuais de um cliente, unindo as tabelas `historico_contratos` e `contratos` em dois *joins* distintos (um para o contrato encerrado e outro para o novo) e exibindo os termos anteriores e posteriores a cada transição (Figura 2), o que comprova o funcionamento do versionamento por *SCD Type 2*.

Figura 2 – Screenshot da tela de histórico de contratos.



Fonte: Autores (2026).

A segunda, Q05, consolida a visão financeira mensal unindo por UNION ALL as receitas provenientes de faturamento recorrente e de parcelas de projetos, fontes que residem em tabelas distintas, produzindo uma linha por mês com a receita total da consultoria (Figura 3). Essa consulta responde diretamente à dificuldade relatada de apurar o faturamento ao fim de cada mês.

Figura 3 – Screenshot da tela de histórico de faturamentos mensais.

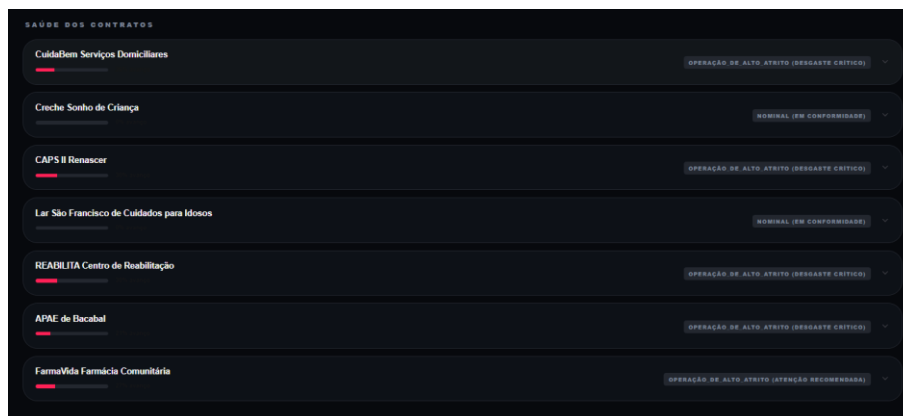
Mês	RECORRENTE	TOTAL FATURADO
Dezembro De 2025	R\$ 51.000	R\$ 51.000
Janeto De 2026	R\$ 51.000	R\$ 51.000
Fevereiro De 2026	R\$ 51.000	R\$ 51.000
Março De 2026	R\$ 51.000	R\$ 51.000
Abril De 2026	R\$ 51.000	R\$ 94.983,33
Maio De 2026	R\$ 51.000	R\$ 94.983,33
Junho De 2026	R\$ 40.000	R\$ 90.483,33
Julho De 2026	R\$ 40.000	R\$ 84.750

Fonte: Autores (2026).

A terceira, Q11, calcula um indicador de saúde operacional por cliente, combinando cinco sinais em um único resultado por contrato ativo: presença de visitas com contexto de emergência, pendências com data limite vencida, eventos críticos sem ação tomada registrada, proporção de entregas concluídas em relação ao total previsto e tempo de inatividade por contrato. A partir desses sinais, cada contrato é classificado em uma de três categorias: “normal”, “atenção” ou “emergência”,

conforme a gravidade dos indicadores identificados (Figura 4). Com os dados de seis meses de operação simulada, a Q11 retornou classificações variadas entre os sete clientes, com alguns em operação normal e outros com pendências atrasadas e eventos críticos sem resposta, o que confirmou que a consulta distingue efetivamente estados operacionais distintos em vez de retornar o mesmo resultado para todos os contratos.

Figura 4 – Screenshot da tela de saúde operacional de cada cliente.



Fonte: Autores (2026).

A Q11 foi a que mais revelou sobre o modelo: na primeira versão, retornava contagens infladas por causa de *joins* cartesianos entre pendências e eventos críticos. O ajuste exigiu reescrever as agregações com LEFT JOIN e contagens distintas. O fato de o problema estar na consulta, não no esquema, confirmou que a estrutura do modelo estava correta.

Outras duas consultas merecem registro, ainda que sem o mesmo nível de complexidade das três anteriores. A Q12 lista a agenda diária do consultor, filtrando visitas e vencimentos de projeto por data, e foi a primeira consulta a revelar um problema de cardinalidade: na versão inicial, visitas vinculadas a projetos apareciam duplicadas por causa de um *join* não filtrado com a tabela projetos. A correção foi simples, com a adição de um LEFT JOIN com filtro explícito, mas o problema confirmou que a agenda é o fluxo mais sensível a *joins* mal posicionados.

A Q14 rastreia pendências em aberto por responsável e prazo, sendo a única consulta que cruza as duas dimensões ao mesmo tempo: se a pendência é do cliente ou do consultor, e se está dentro ou fora do prazo. O resultado é uma visão

que o proprietário não conseguia obter sem busca manual e que, a partir do modelo, passou a ser gerada com uma única consulta.

As quatorze consultas, tomadas em conjunto, funcionaram como um teste de cobertura informal do modelo: cada uma exigiu cruzar pelo menos duas tabelas, e nenhuma exigiu alterar o esquema após a primeira versão estável. Esse resultado não era garantido desde o início: durante o desenvolvimento, três consultas precisaram ser reescritas por problemas de *join*, e uma quarta revelou que a *constraint* de unicidade em `faturamento_cliente` estava mal posicionada e precisou ser ajustada.

O fato de todos esses problemas terem sido resolvidos sem reestruturar entidades ou relacionamentos confirmou que a arquitetura central do modelo era suficiente para o escopo definido. O que ficou fora desse escopo, como múltiplos consultores, notificações automáticas e carga concorrente, não invalida o modelo, mas delimita com precisão o que a validação cobre e o que ainda precisa ser testado.

4 CONSIDERAÇÕES FINAIS

O projeto partiu de um problema real: a AM Consultoria não conseguia fechar o faturamento mensal sem busca manual em fontes fragmentadas, e o histórico de um cliente era impossível de reconstruir sem esforço considerável. A modelagem resolveu esses dois problemas na camada de dados, e a integração com o *frontend* confirmou que o esquema sustenta os fluxos operacionais sem exigir ajustes estruturais.

O levantamento de requisitos revelou que a variabilidade entre contratos era o principal obstáculo à padronização: cada cliente opera com combinações distintas de frequência de visitas, formas de pagamento e escopo de projetos. Essa heterogeneidade orientou a arquitetura do modelo, que priorizou flexibilidade sem abrir mão de rastreabilidade.

Das quinze entidades do modelo, a que concentra mais decisões de projeto é `historico_contratos`. Ela resolve o versionamento, mas aumenta a complexidade de qualquer consulta que precise do estado atual de um contrato, sendo sempre necessário filtrar pelo registro mais recente. Esse *trade-off* foi considerado aceitável dado o requisito de auditabilidade, mas seria o primeiro ponto a revisar se as consultas apresentassem problemas de desempenho em produção. A desnormalização em

faturamento_cliente, por sua vez, emergiu durante a implementação e não foi planejada desde o início, o que mostra que parte das decisões de modelagem foi tomada a partir do que a implementação revelou.

A implementação em PostgreSQL com *migrations* versionadas foi realizada com atenção à integridade no nível do esquema: *constraints* CHECK e UNIQUE foram aplicadas para impedir estados inválidos sem depender exclusivamente de validação no *backend*. As quatorze consultas cobriram os fluxos prioritários do levantamento, mas o que mais surpreendeu durante esse processo foi a relação inversa entre complexidade da consulta e complexidade do esquema: as três consultas mais simples, como agenda diária, listagem de pendências por cliente e contratos próximos do vencimento, foram as que mais revelaram problemas de *join* e precisaram de mais iterações para estabilizar.

As consultas analíticas complexas, como a Q11, foram mais trabalhosas de escrever, mas encontraram o modelo exatamente como esperavam. Isso sugere que os fluxos operacionais cotidianos são o teste mais rigoroso de um modelo de dados, não os casos de uso analíticos que costumam receber mais atenção durante o projeto.

A validação com o banco hidratado cobriu o ciclo operacional completo esperado para uso inicial, mas não constitui teste de carga, já que não foram simulados acessos concorrentes nem volumes acima do porte atual da consultoria. A continuidade mais urgente é a validação dos requisitos com o proprietário real, pois esse passo pode revelar entidades faltantes, cardinalidades erradas ou fluxos não mapeados pelo agente simulado. A partir disso, faz sentido estender o modelo para múltiplos consultores e conectar as consultas de pendências atrasadas e contratos próximos do vencimento a um mecanismo de notificação automática, que já está mapeado no modelo, mas ainda não foi implementado.

REFERÊNCIAS

CAYRES, Paulo Henrique. **Modelagem de banco de dados**. 1. ed. Rio de Janeiro: Escola Superior de Redes, 2015. Disponível em: [https://www.kufunda.net/publicdocs/Modelagem%20de%20banco%20de%20dados%20\(Paulo%20Henrique%20Cayres\).pdf](https://www.kufunda.net/publicdocs/Modelagem%20de%20banco%20de%20dados%20(Paulo%20Henrique%20Cayres).pdf). Acesso em: 19 maio 2026.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson Education do Brasil, 2018.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 7. ed. [S.l.]: Clube de Autores, 2022. ISBN 978-65-01-22210-3. Disponível em: <https://projbd.heuser.pro.br>. Acesso em: 20 maio 2026.

IBM. **Propriedades de operador de dimensão de mudança lenta (SCD)**. In: IBM Documentation, [s.d.]. Disponível em: <https://www.ibm.com/docs/pt-br/db2/11.1.0?topic=operators-slowly-changing-dimension-scd-targetoperator-properties>. Acesso em: 20 maio 2026.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de banco de dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.

SOUSA, Iuri Gregório de. **Economia do cuidado**. Brasília: Câmara dos Deputados, Consultoria Legislativa, 2017. Disponível em: <https://www2.camara.leg.br/atividade-legislativa/estudos-e-notas-tecnicas/fiquePorDentro/temas/economia-do-cuidado-set-2017>. Acesso em: 20 maio 2026.